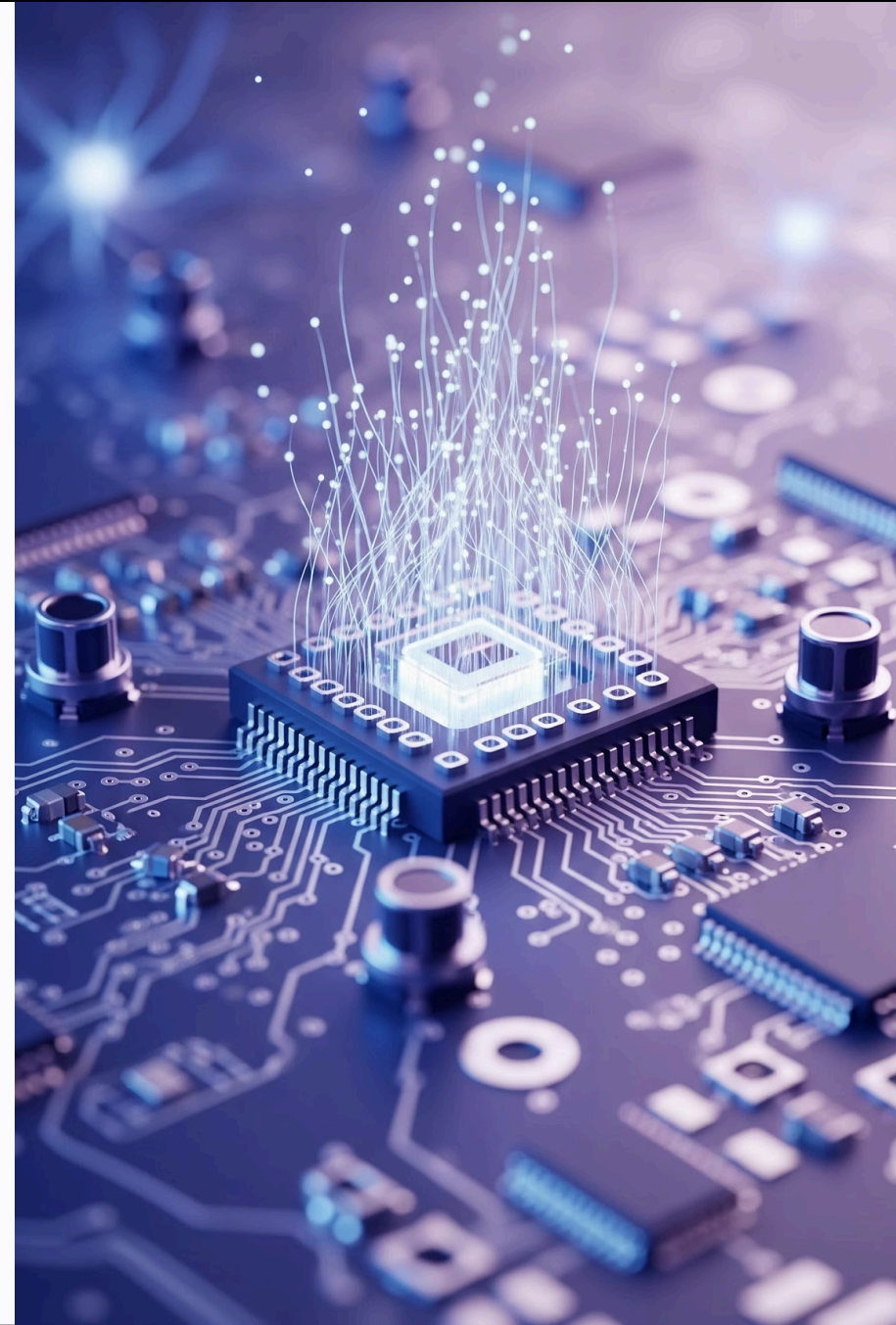
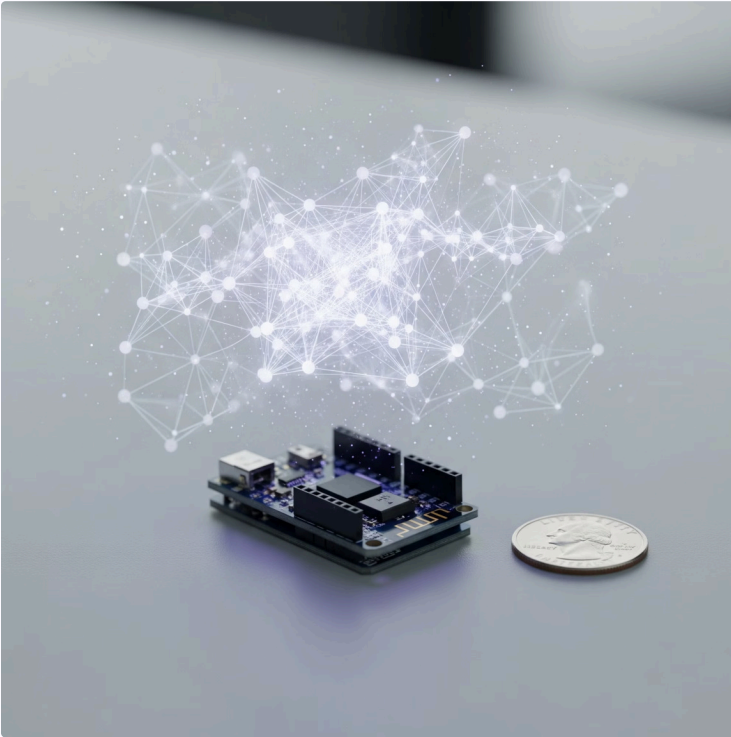


TinyML and Microcontrollers for AI

AI on devices with kilobytes of memory



What Is TinyML?



TinyML brings machine learning capabilities to **microcontrollers (MCUs)** with severely constrained resources:

- Runs on devices with just **KBs–MBs of RAM**
- Enables **always-on AI** for sensors, wearables, IoT
- Excels at specific tasks: keyword spotting, anomaly detection, gesture recognition

Why TinyML Matters



Ultra-Low Power

Operates on batteries for months or even years without recharging, enabling deployment in remote locations



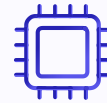
Privacy-First

Processes sensitive data locally on the device, never sending raw information to the cloud



Real-Time Inference

Delivers millisecond-level latency for time-critical applications, without network delays



Massive Scale

Leverages billions of MCUs already deployed worldwide, transforming ordinary devices into smart ones

Hardware Examples



1

Arduino Nano 33 BLE Sense

9-axis IMU, microphone, environmental sensors for ML prototyping

2

STM32 MCUs

Industrial-grade microcontrollers widely used in automation

3

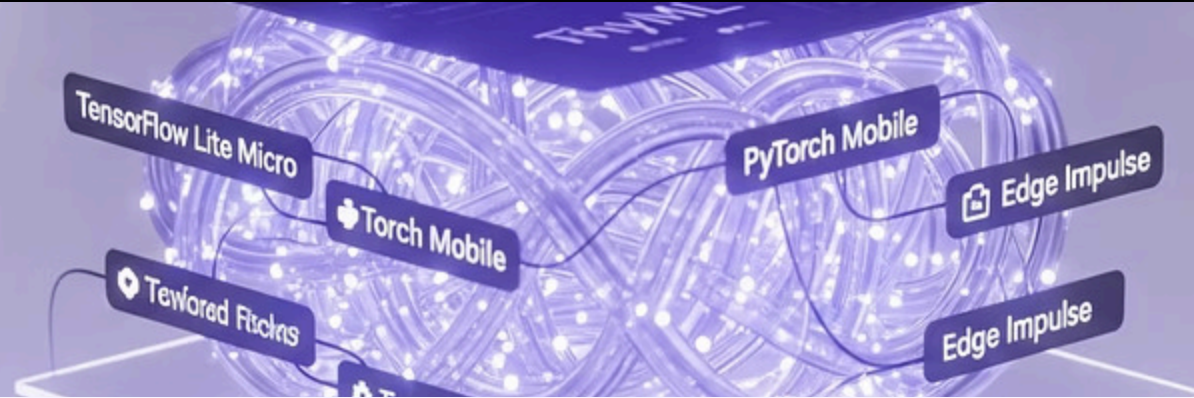
ESP32

Connectivity-focused MCU with Wi-Fi and Bluetooth for IoT applications

4

Edge TPU/NPU

Neural processing co-processors for accelerated inference



TinyML Software Stack

The tools enabling machine learning on resource-constrained devices:

TensorFlow Lite for Microcontrollers

Google's framework for running ML on devices with **as little as 16KB of memory**

Edge Impulse

End-to-end development platform simplifying data collection, training, and deployment to MCUs

CMSIS-NN

ARM's optimized neural network kernels for maximum performance on Cortex-M processors

ONNX Runtime Mobile

Microsoft's lightweight inference engine supporting multiple model formats

Example: Keyword Spotting



Collect Audio Samples

Record 1-second clips of keywords and background noise



Train CNN on Spectrograms

Convert audio to visual patterns for neural network processing



Convert to TFLM Model

Generate .tflite file optimized for microcontrollers



Deploy on Arduino

Flash the model onto Arduino Nano BLE Sense



Test Device Response

LED lights up when keyword is detected



Task: Create a wake-word detection system that runs entirely on a microcontroller with no cloud connectivity required



Example: Anomaly Detection in IoT

The Challenge

Detect equipment failures before they happen, without constant cloud connectivity or heavy power consumption

The TinyML Solution

- Vibration sensor monitors motor operation
- MCU runs **autoencoder model** locally
- Algorithm learns "normal" operation patterns
- Alerts sent only when anomalies detected
- Battery life extended from days to months



Model Optimization Techniques

Making neural networks fit into kilobytes of memory:

1

Quantization

Convert 32-bit floating point to INT8 or even INT4, reducing model size by 4-8x with minimal accuracy loss

2

Pruning & Distillation

Remove unnecessary connections and transfer knowledge from larger models to smaller ones

3

Feature Engineering

Use domain-specific features (MFCC for audio, statistical features for sensors) to reduce input complexity

4

Micro-Architectures

Design specialized tiny networks like micro-CNNs and RNN-lite variants specifically for MCU constraints

TinyML Challenges

Memory Constraints

Working within 32KB–512KB RAM limits requires extreme optimization of both model and runtime

Power Management

Devices must sleep between inferences to conserve power, requiring careful duty cycling

Bare Metal Programming

Many MCUs run without an operating system, complicating development and debugging

Limited Debugging

Restricted logging capabilities make troubleshooting significantly more difficult than cloud environments



⊗ The gap between simulation and real-world deployment can lead to unexpected issues!

Industry Use Cases



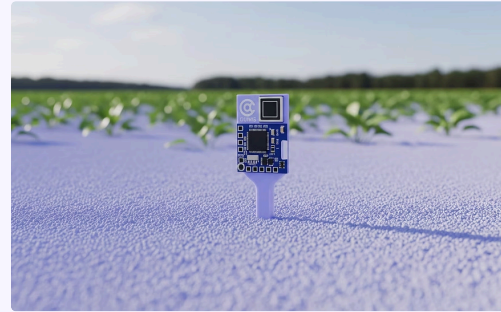
Wearables

Heart-rate anomaly detection, activity classification, sleep analysis



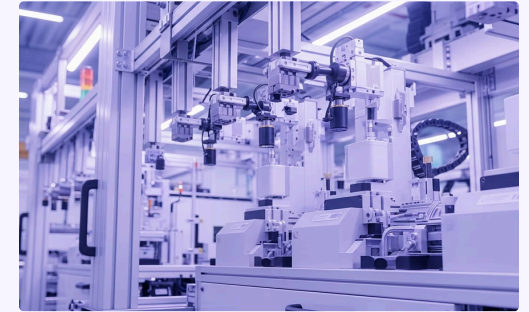
Smart Homes

Wake word detection, gesture recognition, presence detection



Agriculture

Crop monitoring, irrigation optimization, pest detection



Industrial IoT

Predictive maintenance through vibration and sound analysis

All powered by **tiny models running on microcontrollers**, not the cloud

Best Practices



1 Design for Extreme Constraints

Keep models tiny & quantized; aim for <10KB if possible

2 Use Specialized Tools

Edge Impulse and similar platforms simplify the data-to-deployment pipeline

3 Benchmark on Real Hardware

Simulators can't capture true performance; test latency & memory on actual MCUs

4 Optimize Power Consumption

Design for sleep/wake cycles and minimize active processing time

5 Focus on Specific Tasks

Target narrow AI applications rather than general intelligence

Key Takeaways

AI everywhere, even on \$5 microcontrollers

Transformative Benefits

- Real-time, on-device inference
- Enhanced privacy (no cloud required)
- Ultra-low power consumption
- Massive deployment potential

Technical Approach

- Optimized, quantized models
- MCU-friendly runtimes
- Specialized architectures
- Task-specific designs

Getting Started

- Try Arduino Nano 33 BLE Sense
- Experiment with Edge Impulse
- Begin with simple classification
- Join the TinyML community

The future of AI isn't just in the cloud—it's in billions of tiny, intelligent devices all around us